

Qwen3-Coder-Next Tier 3

Part 5 Detailed Implementation Specification

Memory System

Engineering specification for persistent working, session, project, and global memory

Document status

Architecture is already finalized. This specification defines the concrete Memory System implementation, its data contracts, storage tiers, retrieval behavior, and completion criteria.

1. Purpose

The Memory System exists to give the agent durable context across turns, tasks, sessions, and projects. Without it, every request begins as if nothing happened before, which is a spectacularly inefficient way to build a software system.

This part solves four recurring problems: repeated re-discovery of project decisions, loss of bug-fix history, inability to reuse prior task context after restart, and inconsistent retrieval of relevant past knowledge.

Key dependency

Later parts depend on memory to preserve decisions, errors, retrieval context, evaluation results, and recovery history. If memory is weak, every downstream agent becomes noisier and less reliable.

2. Scope

Included	Excluded
Four-tier memory model: Working, Session, Project, Global	Code Knowledge Graph construction
Memory read/write APIs and routing	Context compression policy design outside memory
Persistence, indexing, and retrieval	Testing agent logic and assertions
Session summarization and project decision capture	Full benchmark harness implementation
Retention, promotion, and eviction rules	Deployment and human approval logic
Memory quality verification hooks	Low-level model serving internals

3. System Overview

The Memory System sits beside the orchestrated agent pipeline. It records what happened, retrieves what matters, and exposes a clean memory context to future agents.

Write path:

User/Agent -> Memory Router -> Tier Selection -> Store/Index -> Summary/Metadata

Read path:

User/Agent -> Memory Query -> Tiered Retrieval -> Rank/Filter -> Memory Context -> Agent

Working memory is ephemeral and lives inside the active execution state. Session, Project, and Global memory are durable and queried through explicit APIs. The system must support both immediate recall and long-lived reuse.

4. Internal Architecture

The implementation should be split into small, explicit components. Each component owns one part of the memory lifecycle and communicates through typed records rather than free-form blobs.

Memory Router -> decides tier and operation		
Session Store -> stores conversation outcomes and task history		
Project Store -> stores architecture decisions, fixes, conventions		
Global Store -> stores reusable cross-project patterns		
Retriever -> fetches and ranks candidate memories		
Summarizer / Promoter -> converts raw events into durable memory objects		
Component	Responsibility	Primary dependencies
Memory Router	Selects write/read path and applies policy	Tier rules, request metadata
Working Memory Cache	Holds current task context in runtime state	LangGraph state
Session Store	Persists per-session events and summaries	Local DB or document store
Project Store	Persists project-specific decisions and fixes	Semantic index + metadata store
Global Store	Persists durable reusable patterns	Retention policy + review gate
Retriever	Ranks candidate memories for a query	Embeddings/index + filters
Summarizer	Transforms activity into concise durable memory	Task transcript, outcomes
Policy Engine	Controls retention, promotion, and eviction	Memory classification rules

5. Project Structure

The directory structure should make memory responsibilities obvious. A good memory system is mostly disciplined boundaries and boring filenames. Exactly the sort of thing that saves you from future misery.

Folder	Purpose
--------	---------

memory/	Top-level package for all memory logic
memory/api/	Public read/write interfaces used by agents and orchestration
memory/working/	Runtime-only task context and transient caches
memory/session/	Session lifecycle storage and summaries
memory/project/	Project-scoped durable memory objects
memory/global/	Cross-project reusable memory patterns
memory/index/	Embedding, retrieval, and ranking helpers
memory/policies/	Retention, promotion, and eviction rules
memory/summaries/	Summarization and promotion pipelines
memory/schemas/	Typed data contracts and validation schemas
memory/tests/	Unit, integration, and retrieval benchmark tests
config/	Memory configuration and tier thresholds
observability/	Logging, metrics, and audit events
migrations/	Store schema evolution and backfills

6. Data Contracts

Data contracts must be explicit. Memory failures are usually contract failures dressed up as model problems.

Contract	Essential fields	Example use
MemoryItem	id, tier, subject, content, source, timestamp, confidence, tags	Canonical memory object
MemoryQuery	query_text, tier_hint, project_id, session_id, filters, top_k	Retrieve relevant context
MemoryResult	item, score, rationale, retrieval_source	Return ranked candidates
SessionSummary	session_id, goal, outcomes, decisions, open_questions	Persist session memory
ProjectDecision	project_id, decision, rationale, alternatives, version	Track durable architecture decisions

GlobalPattern	pattern_name, applicability, evidence, caveats	Store reusable best practices
MemoryEvent	event_type, actor, payload, references, status	Append-only activity log
RetentionPolicy	tier, keep_days, promote_on_signal, prune_on_signal	Guides lifecycle rules

Example structured object:

```
MemoryItem:
  id: mem_2026_0017
  tier: project
  subject: 'Use worktrees for risky refactors'
  content: 'Agent modifications must occur in isolated worktrees.'
  source: 'review_orchestration'
  confidence: high
  tags: [worktree, safety, git]
  references: [task_42, commit_abcd1234]
```

7. State Management

Working memory is owned by the runtime and discarded with the active task unless promoted. Session memory is owned by the conversation/session manager. Project and Global memory are owned by the memory service and persisted independently of any single run.

Tier	Lifespan	Owner	Persistence	Consumed by
Working	Active task only	Runtime	In-process state	Planner, coder, tester
Session	Current conversation	Session store	Durable	All agents in the session
Project	Project lifetime	Memory service	Durable	Planner, reviewer, evaluator
Global	Cross-project	Memory service	Durable with stricter gating	Research, planning, summarization

State changes must be append-first. Mutations should be recorded as events, then reflected in derived summaries or indexes. Future agents should consume a stabilized memory_context object rather than raw storage records.

8. Component Specifications

Component	Purpose	Inputs	Outputs	Failure conditions	Future integration
Memory API	Unified read/write interface	Agent requests, task metadata	Normalized memory operations	Schema validation, authz/policy checks	All future agents
Memory Router	Chooses tier and action	Memory requests	Routed operation	Misclassification of memory tier	Evaluation, recovery, repo intelligence
Summarizer	Turns raw sessions into durable records	Transcript, decisions, outcomes	SessionSummary / ProjectDecision	Low-signal summaries, hallucinated compression	Context compression and benchmark harness
Retriever	Finds relevant memories	Query text, filters, top_k	Ranked memory results	Empty results, stale ranking	Planning, research, coding, review
Policy Engine	Controls promotion and eviction	Memory events and thresholds	Action plan	Over-retention, premature pruning	All tiers
Index Manager	Maintains semantic search indexes	MemoryItem writes	Indexed vectors + metadata	Index drift, stale embeddings	Project/global retrieval
Audit Logger	Records memory activity	All writes, reads, promotions	Audit event stream	Missing traceability	Observability and debugging

9. Implementation Sequence

1. Define schemas and validation rules first. Nothing else is trustworthy until the data shape is stable.
2. Implement working memory in runtime state. This gives immediate task context before any persistence work.
3. Add the session store and session summary writer. That establishes replayable history and restart resilience.
4. Implement project memory persistence with metadata and versioning. This makes architectural and bug-fix decisions durable.

- 5. Add retrieval and ranking across tiers. The system must be able to ask for context before it can reuse context.
- 6. Implement promotion and eviction policies. Otherwise the memory store becomes a landfill with embeddings.
- 7. Add global memory gating and stricter promotion rules. Cross-project reuse should be deliberate, not accidental.
- 8. Wire observability and audit trails. Memory without traceability is just confident forgetting.
- 9. Add tests and benchmark hooks. Retrieval quality must be measurable before the system is allowed to depend on it.

10. Validation Strategy

- Unit tests for schema validation, tier routing, and policy decisions.
- Integration tests that write memory, restart the runtime, and verify the records still retrieve correctly.
- Retrieval tests using known queries and known expected memories.
- Regression tests for duplicate summaries, stale memories, and incorrect tier promotion.
- Persistence tests to confirm session and project stores survive process restarts.
- Benchmark tests for retrieval precision, recall, and top-k relevance.

Verification rule

A memory feature is not complete unless an engineer can write a memory, restart the process, query it again, and retrieve the correct result with a visible provenance trail.

11. Deliverables

- Memory API package
- Working memory runtime state model
- Session persistence and summarization pipeline
- Project memory store with versioned records
- Global memory store with promotion gate
- Retrieval and ranking module
- Retention and eviction policy engine
- Audit logging and observability hooks
- Validation tests and retrieval benchmarks
- Configuration templates and operational docs

12. Acceptance Criteria

Criterion	Pass condition
Working memory persists during a single active task	The agent can reuse context after multiple internal steps without re-sending it

Session memory survives restart	A recorded session summary is retrievable after process termination and relaunch
Project decisions are versioned	A changed decision creates a new version, not an overwrite
Bug fixes are retrievable	A query for a known bug returns the documented fix with references
Retrieval quality benchmark passes	Defined top-k relevance threshold is met on the test corpus
Promotion policy is enforced	Only eligible records move from session to project/global tiers
Eviction is deterministic	Records removed by policy are explainably removed and logged
Audit trail is complete	Every memory write and promotion produces a traceable event

13. Common Failure Modes

- Storing everything in one bucket and pretending tiers exist.
- Skipping provenance, which makes retrieved memory impossible to trust.
- Summarizing too aggressively and losing the actual decision or exception.
- Writing memory only on success and forgetting failures, which is where the useful lessons live.
- Letting global memory grow without stricter promotion, review, and retention.
- Using embeddings without metadata filters, which produces vague nonsense at scale.
- Not versioning project decisions, which makes the historical record useless.
- Treating memory as an afterthought instead of a core system boundary.

14. Future Integration

This part is the foundation for later systems that need continuity.

Future part	How it uses memory
Part 6 - Testing & Review	Loads prior bug fixes, known failures, and session outcomes
Part 7 - Evaluation Layer	Compares task outputs against remembered requirements and prior decisions
Part 8 - Failure Recovery	Retrieves historic failure patterns and successful retries
Part 9 - Repository Intelligence	Attaches memory to codebase facts and implementation history

Part 11 - Context Compression	Uses summaries and retrieval results to stay within context limits
Part 12 - Multi-Agent Coordination	Shares a consistent memory substrate between agents
Part 13 - Benchmark Harness	Measures retrieval quality and memory stability over time

15. Completion Checklist

- ☐ Schemas are defined and validated.
- ☐ Working memory exists in runtime state.
- ☐ Session persistence works across restart.
- ☐ Project memory stores versioned decisions and fixes.
- ☐ Global memory promotion is gated and logged.
- ☐ Retrieval returns ranked, filtered results with provenance.
- ☐ Retention and eviction policies are implemented.
- ☐ Audit logs capture every memory mutation.
- ☐ Integration tests pass for write, read, promote, and prune paths.
- ☐ Retrieval benchmark meets the documented threshold.